Microcode State Engine Simulation Analysis

Here is the structured Page artifact synthesized from the group chat:

Research Brief: Microcode State Engine Simulation Analysis

Research Question

How do pointer phase mapping, attractor classification, wave interference, and multi-ring toroidal structures interact to produce stable, periodic, quasi-periodic, or chaotic behaviors in a microcode state engine?

Background Context

The discussion centers on a toroidal memory-process engine composed of three rings (R1, R2, R3), each with eight slots and distinct pointer speeds (1, 2, and 4 steps per cycle). Each slot contains microcode rules: Forced Overwrite, Passive Collapse, or Protected History.
The engine's emergent behaviors are driven by pointer phase mapping, harmonic alignment across rings, and interference between overwrite, collapse, and invariant waves.

Hypotheses / Key Ideas

- **Pointer Phase Mapping**: Phase is a geometric-temporal index combining position and timing. It determines which microcode rule fires and when, creating phase-dependent physics.
- **Attractor Classification**: Four classes exist - static, periodic, quasi-periodic, and chaotic - depending on pointer recurrence and microcode interactions.
- **Wave Interference**: Overwrite, collapse, and protected-history waves interact constructively, destructively, or through anchoring, shaping attractors and invariants.
- **Multi-Ring Harmonics**: The least common multiple of pointer cycles (8) creates harmonic nodes where all rings align, reinforcing invariants and attractors.
- **System Visualization**: ASCII diagrams of rings, pointers, and microcode distributions provide a concrete anchor for understanding system geometry and behavior.

Evidence & References

- **Pointer Phase Mapping**: Demonstrated through cycle-by-cycle phase lattice examples.
- **Attractor Classification**: Defined with causes, examples, and stability levels.
- **Wave Interference**: Illustrated with ASCII overlays showing constructive, destructive, and anchored interference.

- **System Overview Diagram**: High-level ASCII representation of rings, microcode, pointers, and behaviors.
- **Licensing Framework**: Dual license notice, contributor policy, and commercial terms drafted to govern repository use.

Next Steps / Experiments

- Develop cycle-by-cycle timelines of system evolution over 8 cycles.
- Create composite invariant diagrams showing reinforcement across rings.
- Extend attractor classification with quantitative metrics (e.g., cycle length, stability index).
- Explore chaotic attractor conditions through simulation.
- Refine licensing terms with legal counsel for commercial deployment.

Appendix

Licensing Text Blocks

- **Dual License Notice**: Declares MIT coverage for code and documentation, with patent-governed restrictions on commercial use.
- **Dual License Overview (README)**: Public explanation of open-source vs commercial rights.
- **Contributor License Notice**: Clarifies contributor obligations under MIT and patent restrictions.
- **Commercial Licensing Terms Draft**: Outlines rights, restrictions, and contact for commercial licensing.

Visual Artifacts

- Multi-ring toroidal ASCII diagrams (outer, middle, inner rings).
- Wave interference ASCII overlays.
- System overview ASCII diagram summarizing geometry, pointers, microcode, and behaviors.

Action Items

- Update repository with synthesized documentation and diagrams.
- Insert dual license notice into LICENSE file.
- Add overview and contributor notice to README/CONTRIBUTING.md.
- Create LICENSE-COMMERCIAL or /legal folder with draft terms.

- Switch repository visibility back to public after updates.

This Research Brief consolidates the technical exploration of the microcode state engine and the licensing framework for its repository, providing a clear foundation for further development, documentation, and public release.